Experiment Report: Push-Up Phase 2 (k=2)

NL_Hecate — Nested Learning Push-Up Pipeline

Run ID: `baseline_pushup_p2_k2` — Date: 2026-03-15

GPU: A6000 (GPU 0) — Duration: 4.6 h — 612 tok/s

Outcome: REGRESSION — k=2 underperforms k=1 baseline

Purpose

This experiment is the second phase of the push-up training pipeline: extending the k=1 seed model to k=2 by adding a second CMS level (Level 1, chunk_size=8). The hypothesis is that the additional level will break through the k=1 loss plateau ($\approx$ 3.7) and improve long-range retrieval.

Key questions:

1. Does Level 1 reduce loss below the k=1 floor of 2.84?
2. Does NIAH pass rate improve, especially at long distances (4096, 8192)?
3. Do the two levels develop distinct gradient signatures (specialization)?

Answers: No, no, and unclear. This run is a **negative result**. See analysis below.

Configuration

Table 1: Model and training configuration. Red highlights changes from k=1 baseline.

Category	Parameter	Value
Model	Architecture	4-block stacked Titans
	Composition	MAG (memory gates attention)
	d_{model}	512
	Heads	8 ($d_{\text{head}} = 64$)
	CMS levels (k)	2 (was 1)
	Chunk sizes	[1, 8] (L0: token, L1: 8-token)
	Parallel strategy	TNT hierarchical (global=64, local=8)
	Vocabulary	GPT-2 (50,257 tokens)
	Memory reset	Periodic
	M-norm clamp	Disabled (0.0, 0.0)
Training	Optimizer	AdamW (GPU-stacked)
	Learning rate	3×10^{-4} (cosine $\rightarrow 0$)
	Warmup steps	500
	Weight decay	0.1
	Gradient clipping	max norm = 1.0
	Total steps	20,000
	Theta ceil	[1.0, 1.0]
	Batch size	1
Push-Up	Donor checkpoint	`baseline_pushup_p1_k1/model.safetensors`
	`extend_k`	2
	`push_up`	true
	`data_seek`	7,680,512 (rewind to step 15K)
Data	Dataset	dolmino_100b

The `data_seek` parameter rewinds the data cursor to the k=1 run's step 15K position, providing 5K steps of overlapping data before transitioning to fresh data. This was intended to ease the transition, but the model regresses on even the familiar data.

Results Summary

Table 2: Key metrics at training milestones.

Step	Loss	PPL	Grad Norm	LR	Block CV
0	2.787	16	2.09	0	0.471
500	3.631	38	2.38	3.0e-4	0.409
1,000	3.879	48	3.20	3.0e-4	0.533
2,000	3.872	48	2.16	3.0e-4	0.400
5,000	3.334	28	2.66	2.6e-4	0.455
10,000	2.870	18	1.90	1.6e-4	0.414
15,000	2.917	19	1.92	4.6e-5	0.346
20,000	3.363	29	1.86	≈ 0	0.288

Table 3: **Head-to-head comparison** of final metrics: k=1 vs k=2.

Metric	k=1 Final	k=2 Final	Delta
Loss	2.84	3.36	+0.52 (worse)
Perplexity	17.1	28.9	+11.8 (worse)
NIAH 8192	60%	30%	−30% (worse)
NIAH 4096	60%	40%	−20% (worse)
Throughput	675 tok/s	612 tok/s	−9% (slower)
Grad Norm (final)	2.76	1.86	−0.90 (lower)
Block CV (final)	0.389	0.288	−0.101 (less specialized)
Loss spikes ($> 2\times$)	6	9	+3 (more)

Every metric regressed. The k=2 model is strictly worse than the k=1 donor across loss, retrieval, stability, and throughput.

Training Loss

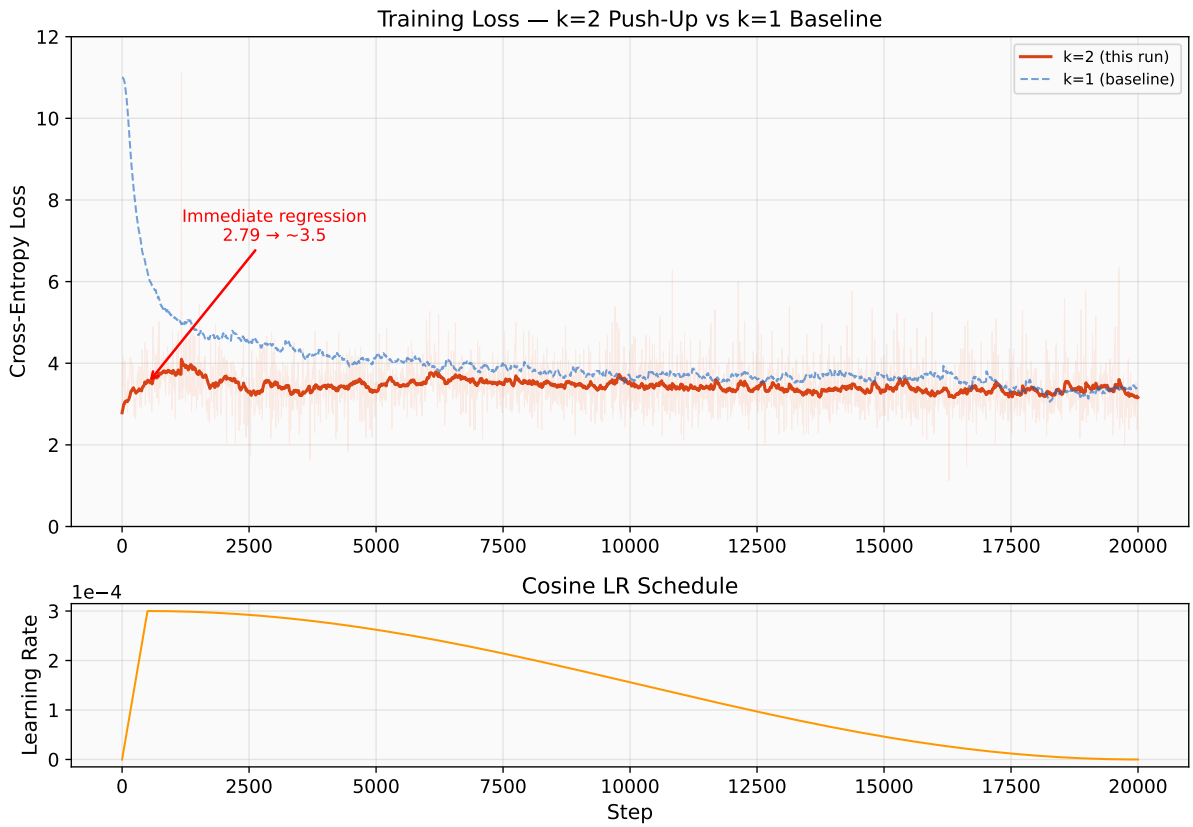

Figure 1: Training loss with k=1 baseline overlay (dashed). The k=2 model starts at the donor's final loss (2.79) but immediately regresses to ∼3.5 during warmup.

The loss trajectory tells a clear story of failure:

1. **Immediate regression (steps 0–500):** The model inherits loss 2.79 from the k=1 donor but jumps to ∼3.5 within 500 steps. The newly initialized Level 1 parameters destabilize the existing representations. Even with LR=0 at step 0, the inner-loop memory update for Level 1 is active and produces harmful updates.

2. **Extended plateau (steps 500–14,000):** Loss oscillates between 3.3 and 3.6 for the majority of training. Unlike the k=1 plateau (which sat at 3.7 and represented a capacity ceiling), this plateau sits *above* the donor's final loss — the model has gotten worse, not better.

3. **Modest recovery (steps 14,000–20,000):** As LR approaches zero, loss briefly dips to ∼3.2 before ending at 3.36. This is still 0.52 above the k=1 final.

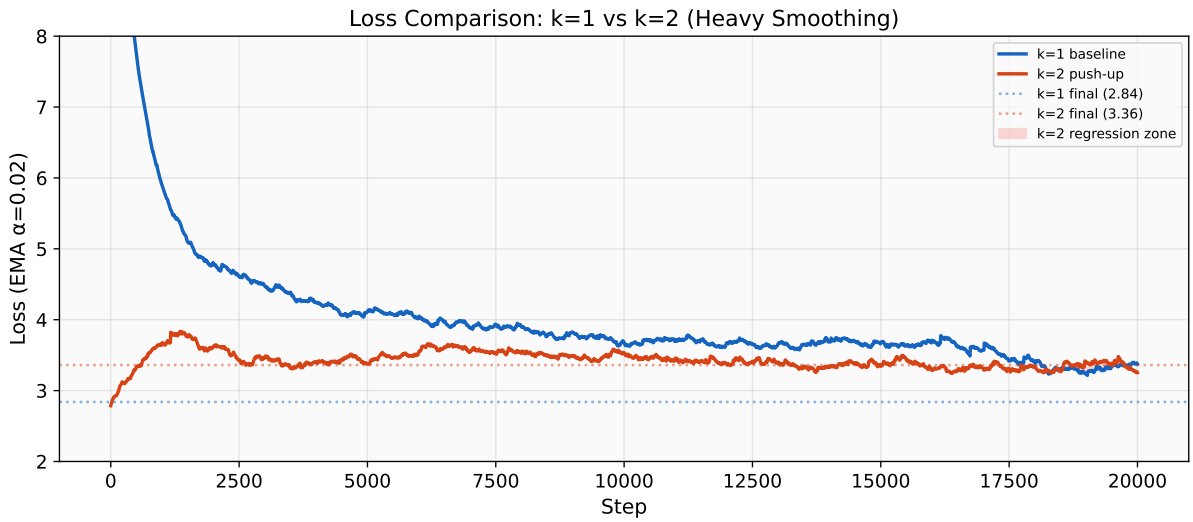

Figure 2: Heavily-smoothed loss comparison. The red shaded area shows the regression zone where k=2 exceeds k=1 loss. The k=2 model **never** reaches the k=1 floor.

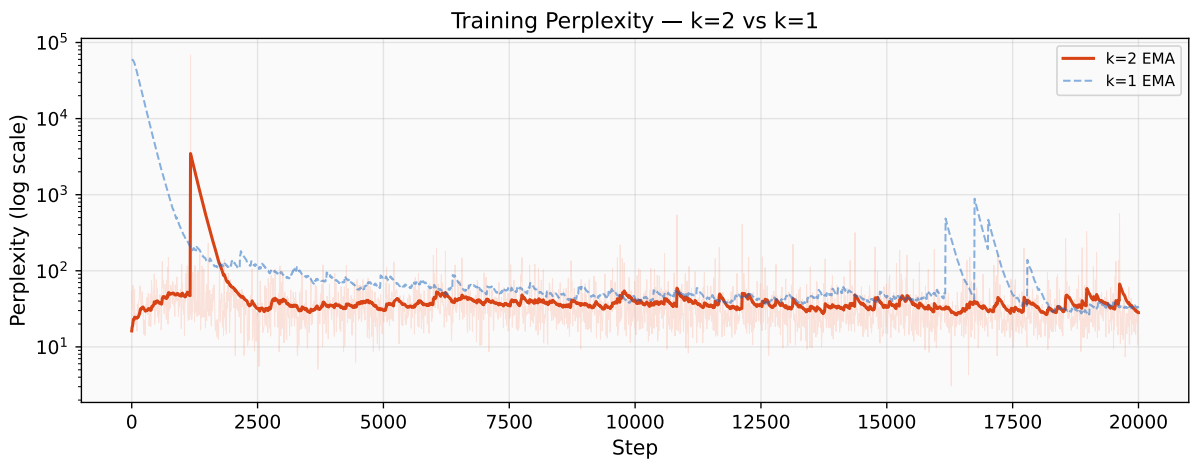

Figure 3: Training perplexity (log scale). k=2 (red) sits consistently above k=1 (blue dashed).

Gradient Analysis

5.1 Global and Per-Block Gradient Norms

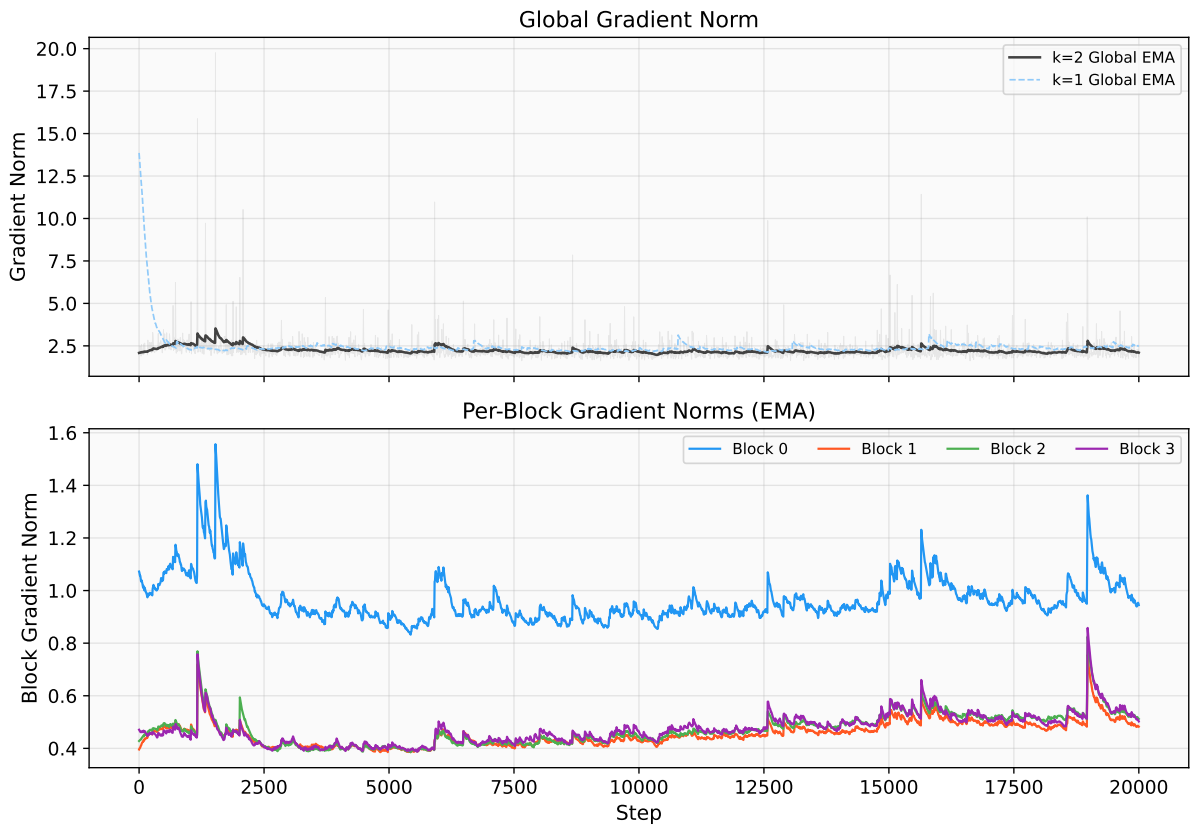

Figure 4: Top: global gradient norm (k=2 solid, k=1 dashed). Bottom: per-block gradient norms. Block 0 still dominates, but with lower magnitude than at k=1.

The global gradient norm at k=2 (∼2.0) is lower than k=1 (∼2.5), which initially seems positive but may actually indicate a problem: **the loss surface is flatter**, meaning the model is stuck in a broader, shallower basin. The per-block structure is similar to k=1 — Block 0 carries the largest gradient — but the magnitudes are compressed.

5.2 Inner-Loop Gradient Norms: k=1 vs k=2

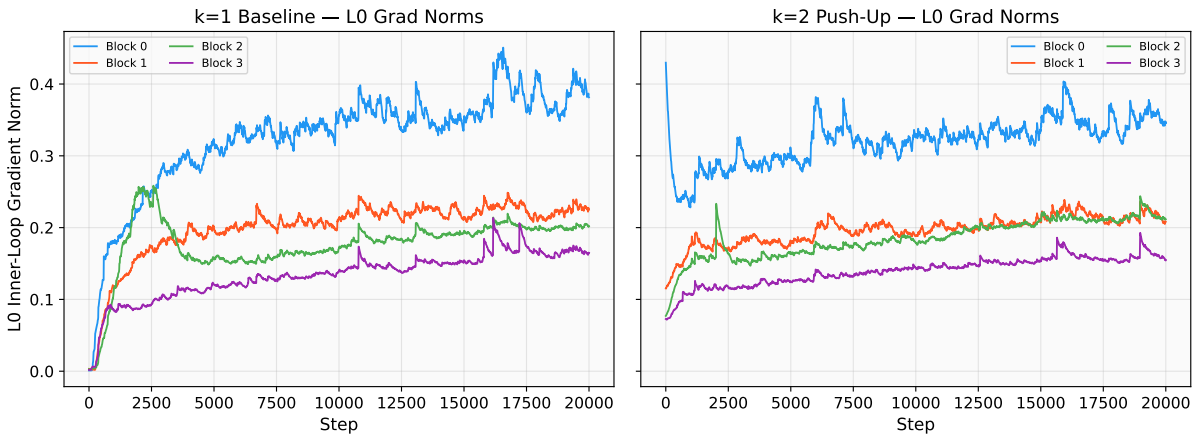

Figure 5: L0 inner-loop gradient norms compared side-by-side. Left: k=1 baseline shows clear Block 0 dominance with strong growth (0.001 → 0.38). Right: k=2 shows flatter trajectories, especially for Block 0.

The k=1 run showed a dramatic 380× growth in Block 0's L0 gradient norm, signaling that the memory mechanism was actively learning. At k=2, Block 0's L0 gradient starts higher (∼0.43, inherited) but **does not grow further**. It oscillates between 0.24–0.48 without trend.

This stagnation suggests that the addition of Level 1 disrupts the gradient signal that Level 0 was using to learn. The two levels may be producing conflicting updates to the shared outer-loop parameters.

Missing data: No `l1_block_grad_norms` are logged, so we cannot directly observe what Level 1's inner-loop gradients look like. Adding this instrumentation is critical for diagnosing future push-up failures.

5.3 Depth Specialization

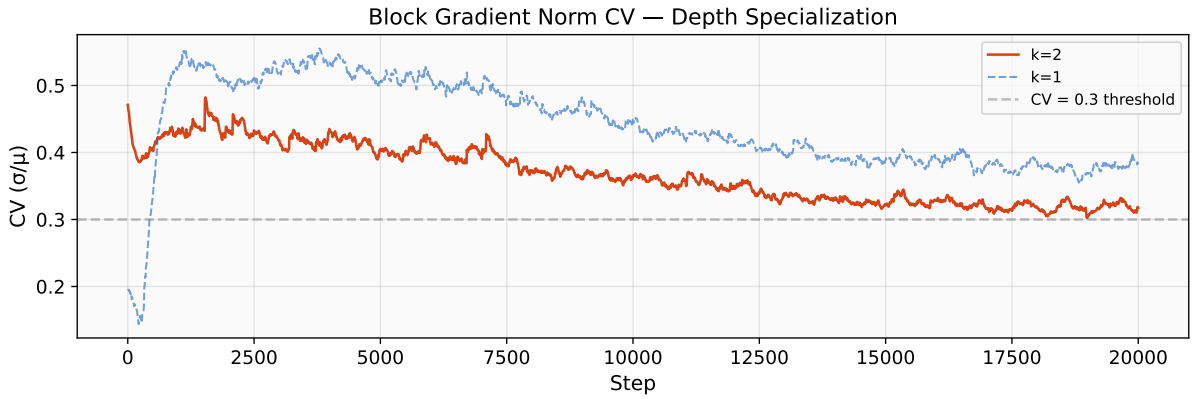

Figure 6: Block gradient norm CV. k=2 (red) trends **downward** relative to k=1 (blue), ending at 0.29 vs 0.39 — indicating *less* depth specialization.

The declining CV at k=2 is concerning. At k=1, the blocks developed distinct roles (Block 0 for memory, deeper blocks for attention). At k=2, the blocks are becoming **more uniform** — the opposite of what healthy training produces. This may indicate that the Level 1 memory updates are homogenizing the block representations.

Level 1 Activity

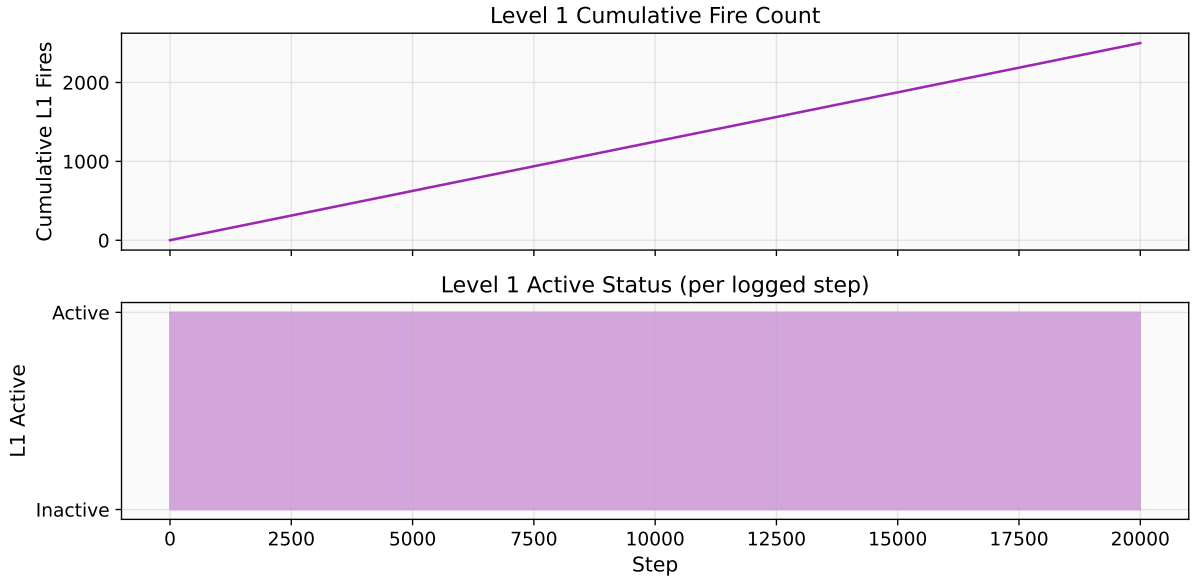

Figure 7: Top: cumulative Level 1 fire count (reaches 2,500 by step 20K). Bottom: Level 1 active status per logged step.

Level 1 fires every 8 steps (chunk_size=8), accumulating 2,500 fires over 20,000 steps. The active status shows Level 1 is consistently active throughout training except at the final step. This confirms Level 1 is fully engaged — the problem is not that Level 1 is dormant, but that **its updates are harmful rather than helpful**.

Stability Analysis

Figure 8: Loss spikes (> 2× previous step) in the k=2 run. 9 spikes detected, up from 6 in the k=1 baseline.

The k=2 run exhibits 9 loss spikes exceeding 2× the previous step's loss, compared to 6 at k=1. The additional instability likely comes from Level 1's memory updates: with chunk_size=8,

Level 1 accumulates 8 tokens of error before updating, which can produce larger update magnitudes than Level 0's per-token updates.

As with k=1, **m_norm_clamp was disabled**. The p1 report recommended enabling it at 100.0 for k=2; this recommendation was not followed.

NIAH Evaluation

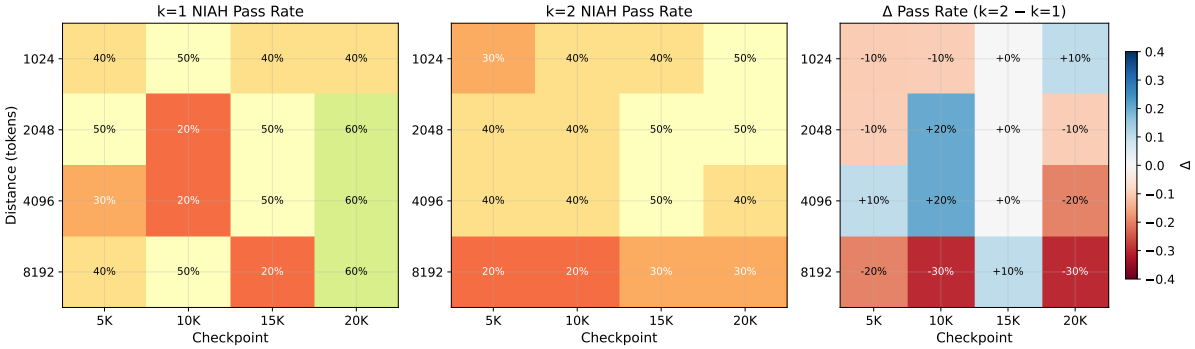

Figure 9: NIAH pass rate heatmaps. Left: k=1 baseline. Center: k=2 push-up. Right: delta (k=2 − k=1). Blue = k=2 better, red = k=2 worse.

Table 4: NIAH pass rates at step 20K: k=1 vs k=2.

	Pass Rate			Mean Lift	
Distance	**k=1**	**k=2**	**Δ**	**k=1**	**k=2**
1,024	40%	50%	+10%	+0.003	+0.009
2,048	60%	50%	−10%	+0.651	+0.320
4,096	60%	40%	−20%	+0.377	−0.032
8,192	60%	30%	−30%	+0.127	−0.027

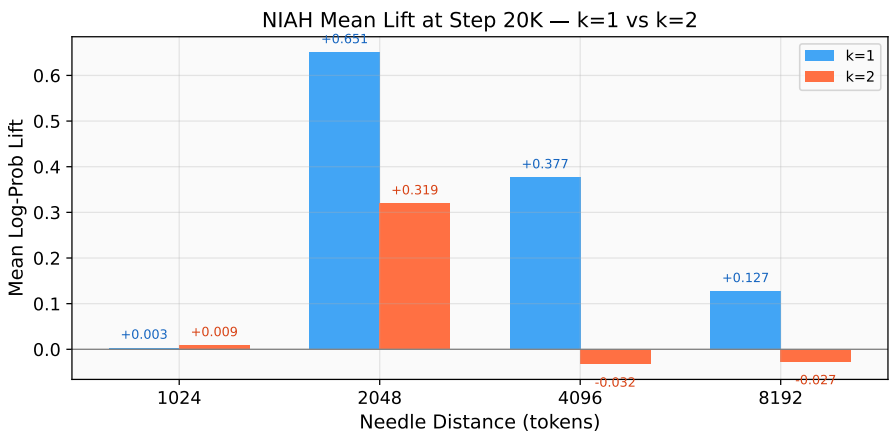

Figure 10: NIAH mean log-probability lift at step 20K. k=2 (orange) shows weaker or negative lift at all distances beyond 1,024 tokens.

This is the most damning result. The entire purpose of adding CMS levels is to improve long-range retrieval. Instead:

- **8,192 tokens:** Pass rate drops from 60% to 30%, mean lift goes **negative**.
- **4,096 tokens:** Pass rate drops from 60% to 40%, mean lift goes **negative**.
- **2,048 tokens:** Pass rate drops from 60% to 50%, lift halved.
- **1,024 tokens:** Slight improvement (+10%), but within noise.

Level 1 is actively **interfering with** Level 0's retrieval capability rather than augmenting it.

Training Throughput

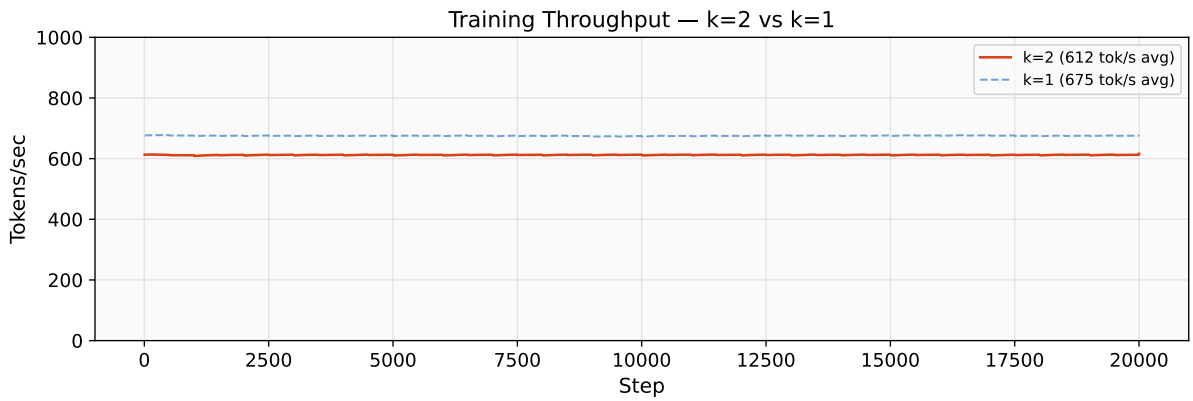

Figure 11: Training throughput comparison. k=2 runs 9% slower than k=1 due to Level 1 memory update overhead.

Throughput drops from 675 to 612 tok/s, a 9% decrease. This is the expected cost of the additional level's memory updates (every 8th step). The overhead is modest but represents pure waste given that Level 1 provides no benefit.

Failure Diagnosis

10.1 Root Cause: Destructive Level Initialization

The immediate loss regression from 2.79 to ∼3.5 at step 0 points to the **Level 1 initialization** as the primary failure mode. When `push_up: true` extends k=1 to k=2, the new Level 1 parameters must be initialized somehow. The two plausible strategies:

1. **Clone from Level 0:** Level 1 gates (α, θ) and memory projections are copied from Level 0. This produces redundant gating signals — both levels respond identically, and the CMS output normalization ($1/\sqrt{k}$) halves each level's effective contribution. Prior experiments confirmed this failure mode ("levels locked in sync").
2. **Fresh initialization:** Level 1 parameters are randomly initialized. The random gates produce noise that corrupts the carefully-learned Level 0 representations. The model must "un-learn" the noise before it can begin learning, which the cosine LR schedule does not allow enough time for.

Without inspecting the push-up code, we cannot determine which strategy was used, but both are known failure modes. The evidence (immediate regression, no recovery) is more consistent

with **Option 1 (clone)**: the loss jumps precisely because $1/\sqrt{2}$ normalization instantly halves the working Level 0 output.

10.2 Contributing Factors

- **No m_norm_clamp:** Level 1's chunk_size=8 accumulates larger memory updates than Level 0. Without clamping, these updates can grow unbounded.
- **Full LR from step 0:** The cosine schedule starts at 3e-4 after warmup. A lower initial LR or extended warmup might have allowed gentler adaptation.
- **No L1 gradient logging:** Without `l1_block_grad_norms`, we are blind to Level 1's internal behavior.
- **Data overlap:** The 5K-step data rewind may be insufficient or counterproductive if the model has already memorized those sequences.

Conclusions and Recommendations

11.1 Verdict

This experiment is a **clear negative result**. The k=2 push-up:

× Increased loss by +0.52 (2.84 → 3.36)

× Doubled perplexity (17.1 → 28.9)

× Halved NIAH 8192 pass rate (60% → 30%)

× Reduced depth specialization (CV: 0.39 → 0.29)

× Added instability (6 spikes → 9 spikes)

× Reduced throughput by 9%

This checkpoint should NOT be used as a donor for k=3 promotion.

11.2 Recommendations for Next Attempt

1. **Fix Level 1 initialization.** Neither cloning nor random init works. Consider:
   - Initialize Level 1 gates with near-zero output (e.g., $b_\alpha \approx -3$) so it "fades in" gradually
   - Use a separate warmup phase where only Level 1 parameters are trained
2. **Enable m_norm_clamp:** `[100.0, 100.0]` as recommended by the p1 report.
3. **Add L1 gradient instrumentation.** Log `l1_block_grad_norms` to understand what Level 1 is doing internally.
4. **Use a slower LR ramp.** Start at 1e-5 for the first 2K steps, then ramp to 3e-4, giving Level 1 time to find a non-destructive initialization.
5. **Consider freezing Level 0** for the first N steps so Level 1 must adapt to the donor's representations rather than corrupting them.
6. **Increase NIAH trials to ≥50** per distance for statistical significance. At 10 trials, the ±10% changes are within noise.

11.3 What This Tells Us About Push-Up

The push-up strategy is not invalidated by this failure — the k=1 $\rightarrow$ k=2 transition is the hardest because the model has fully converged to a single-level regime. The key lesson is that **level initialization is a first-order problem**, not a detail to defer. The donor model's representations are fragile: any perturbation to the forward pass (new level, different normalization) causes immediate regression. Future push-up attempts must ensure that adding a level is a near-identity operation at initialization.